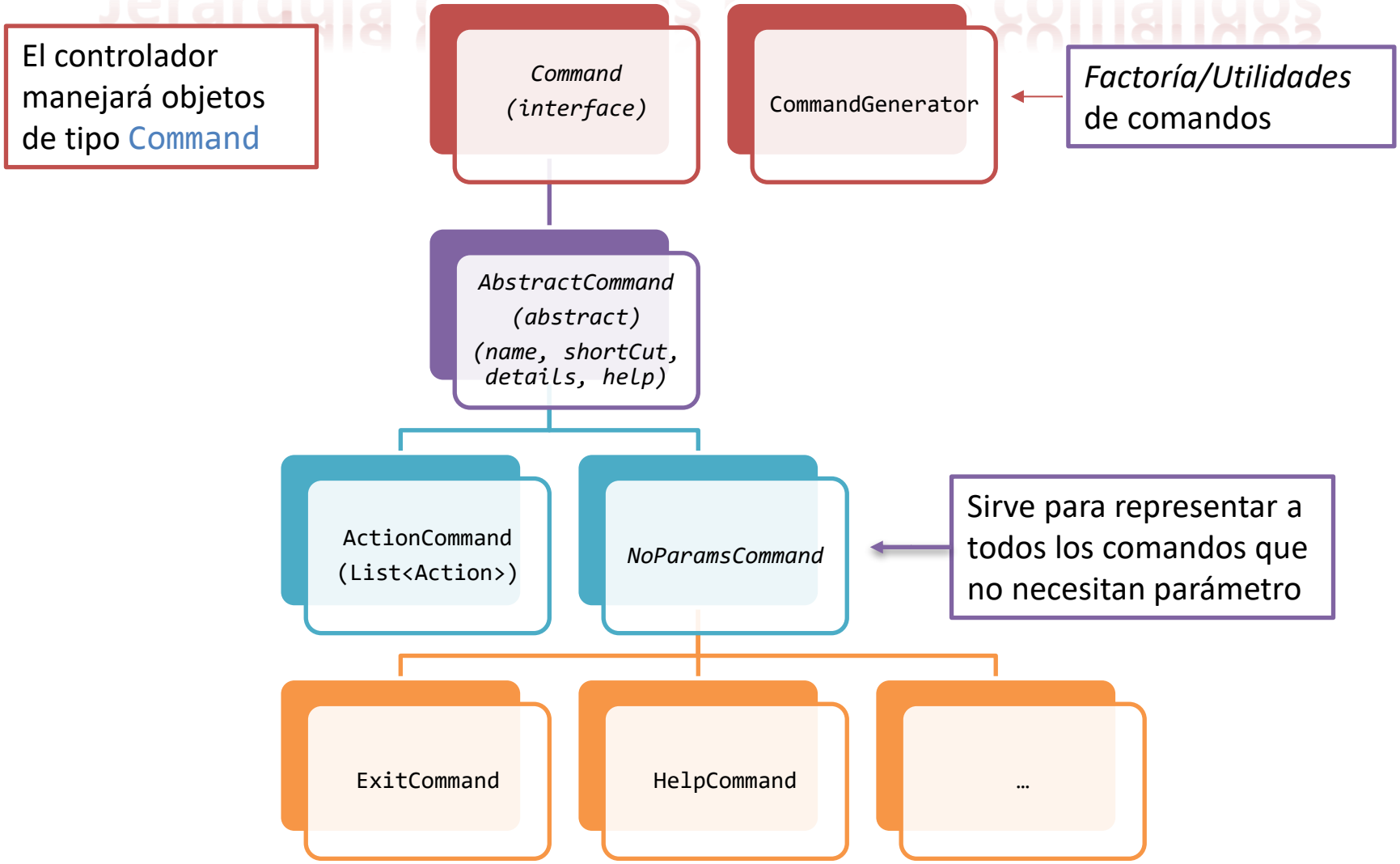
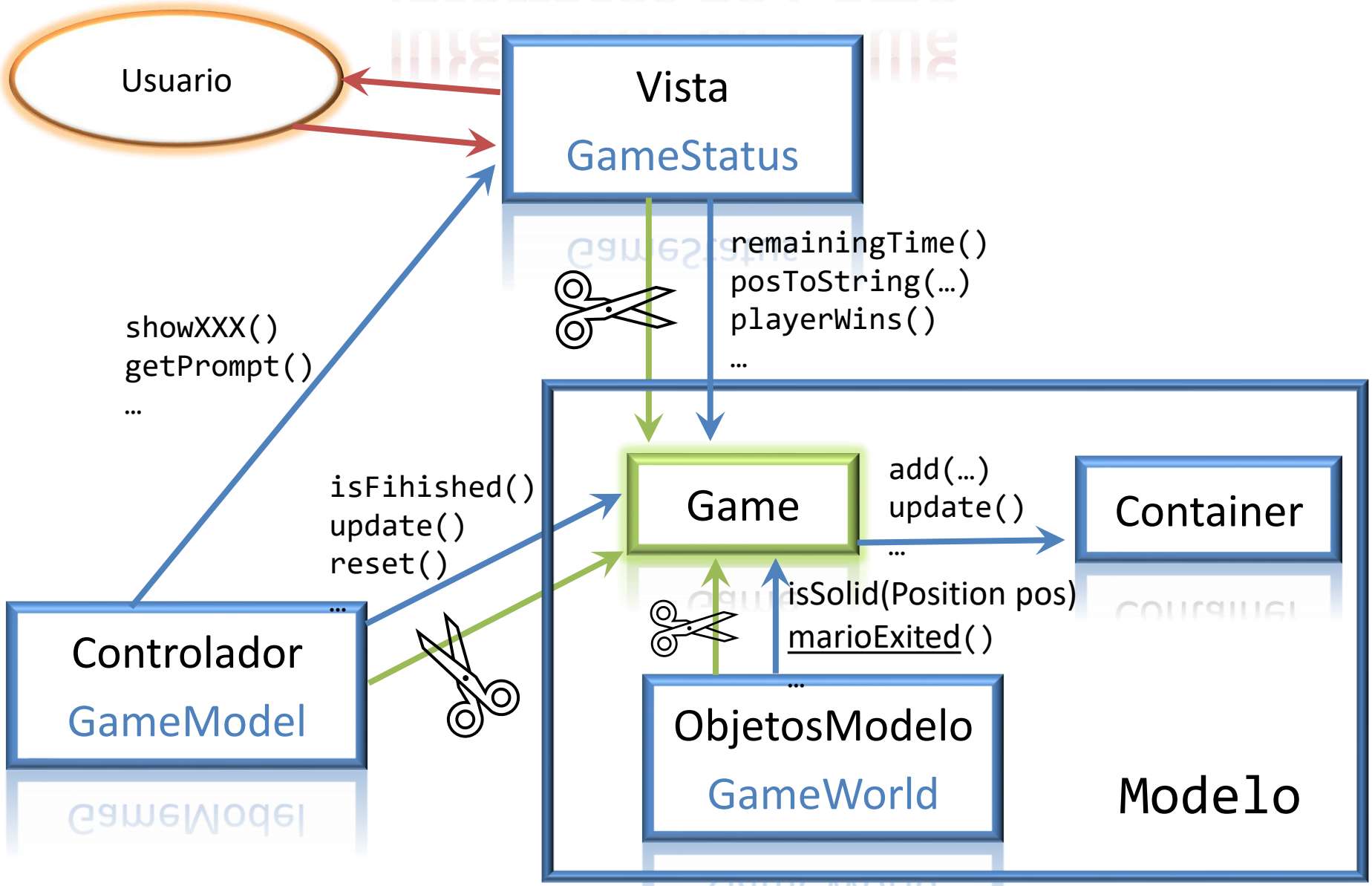
Práctica 2

Jerarquía de clases para los comandos



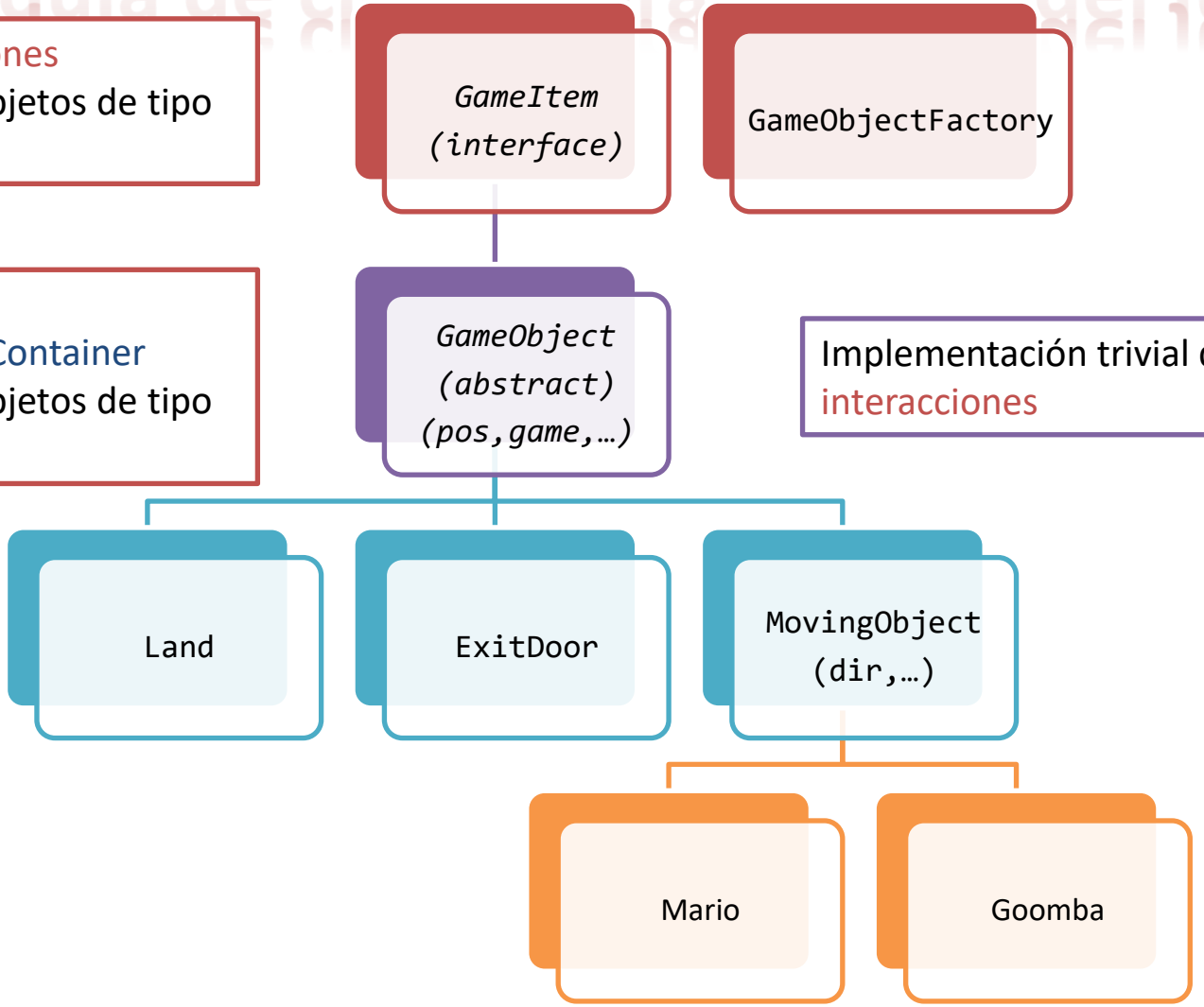
Interfaces de Game



Jerarquía de clases para objetos del juego

Las interacciones manejarán objetos de tipo GameItem

Game y GameObjectContainer manejarán objetos de tipo GameObject



Implementación trivial de las interacciones

Beneficios:

- Manejar los `GameObject` como una abstracción permite utilizar una única estructura de datos para almacenarlos, en lugar de tres listas

Contenedor de objetos del juego

Clase única contenedora de objetos

```
public class GameObjectContainer {
  private List<GameObject> objects;
  public GameObjectContainer() { objects = new ArrayList<>(); }

  public void add(GameObject object) { objects.add(object); }

  public void update() {
    for (GameObject object : objects)
      object.update();
        …
    removeDead();
  }

}
```

Usamos una colección genérica de Java

Polimorfismo

Vinculación dinámica

¿Qué pasa si se modifica la lista mientras se recorre?

Beneficios:

- Encapsulación de los métodos relacionados con las interacciones

Generalizado las interacciones: Interfaz `GameItem` de `GameObject`

Generalizando las interacciones

- Todos los objetos del juego deben tener la posibilidad de recibir interacciones de otros objetos
- Problema:
  - Los objetos se almacenan de modo abstracto, como *GameObjects* (¡y así queremos que sea!)
  - Pero el resultado de las interacciones depende de los tipos concretos:
    - Mario puede matar a un Goomba o ser matado por un Goomba…
- **Intento** de solución: en *GameObject* podríamos declarar

```
public abstract boolean interactWith(GameObject obj);
```

  - Cada subclase de *GameObject* puede implementarlo a su gusto.
  - La vinculación dinámica permite elegir un método dinámicamente
  - PERO: no permite elegir un método en función de los tipos de los argumentos

NECESITAMOS LA “MATRIZ” DE LAS INTERACCIONES

Interfaz para las interacciones

- Definimos un interfaz para las interacciones (y más cosas)

```
public interface GameItem {

   public boolean isSolid();
   public boolean isAlive();
   public boolean isInPosition(Position pos);

   // They usually have to be in the same position to interact.
   public boolean interactWith(GameItem item);

   public boolean receiveInteraction(Land obj);
   public boolean receiveInteraction(Mario obj);
   public boolean receiveInteraction(ExitDoor exitDoor);
   public boolean receiveInteraction(Goomba obj);
}
```

```
public abstract class GameObject implements GameItem
```

Técnica de double-dispatch

- Cada clase concreta tiene que tener su propia implementación de `interactWith(GameItem other)`

```
public class Land extends GameObject {

  ……
  public boolean interacWith(GameItem other) {
        … other.receiveInteraction(this);
  }

}
```

Tipo estático de **this**: Land

Método invocado: receiveInteraction(Land)

- La resolución de la sobrecarga la realiza el compilador usando el tipo estático

Parte II: Extensiones

Nuevo comando AddObjectCommand

```
Command > h
[DEBUG] Executing: h

Available commands:
   [a]dd[O]bject <object_description>: adds to the board the object given by object_description
   [n]one | "": user does not perform any action
   [r]eset: reset the game to initial configuration
   [h]elp: print this help message
   [e]xit: exits the game
```

Mushroom

- Se mueve igual que un Goomba
- Con dirección inicial hacia la derecha
- Interacciona con Mario haciéndolo crecer

Box

- Objeto solido
- Al colisionar Mario desde abajo hace aparecer un Mushroom encima de él
- Tras la primera colisión quedará vacío